Informed Search: A*, GBF, UCS

Notebook version of mazeRunnerFinished.py

In [4]:
```
import random
import numpy as np
class Node():
    """A node class for A* Pathfinding"""

    def __init__(self, parent=None, position=None):
        self.parent = parent
        self.position = position

        self.g = 0
        self.h = 0
        self.f = 0

    def __eq__(self, other):
        return self.position == other.position

    def __hash__(self):               #<-- added a hash method
        return hash(self.position)

# method ='AStar', 'GBF', 'UCS'
# 'AStar': A-star search, 'GBF': greedy best first, 'UCS': uniform cost sear
def InformedSearch(maze, start, end, method='Astar'):
    """Returns a list of tuples as a path from the given start to the given

    # Create start and end node
    start_node = Node(None, start)
    start_node.g = start_node.h = start_node.f = 0
    end_node = Node(None, end)
    end_node.g = end_node.h = end_node.f = 0

    # Initialize both open and closed list
    open_list = []
    closed_list = set()                # <-- closed_list must be a set

    # Add the start node
    open_list.append(start_node)

    # Loop until you find the end
    expanded_nodes=0
    queue_size=0
    while len(open_list) > 0:

        # Get the current node
        current_node = open_list[0]
        current_index = 0
        for index, item in enumerate(open_list):
            if item.f < current_node.f:
                current_node = item
```

```
                current_index = index

        # Pop current off open list, add to closed list
        open_list.pop(current_index)
        closed_list.add(current_node)     # <-- change append to add

        # Found the goal
        if current_node == end_node:
            path = []
            current = current_node
            while current is not None:
                path.append(current.position)
                current = current.parent
            return(expanded_nodes,queue_size,path[::-1]) # Return reversed p

        # update expanded nodes, and update maximum queuze size
        expanded_nodes=expanded_nodes+1
        if(len(open_list)>queue_size):
            queue_size=len(open_list)  # check maximum queue size

        # Generate children
        children = []
        for new_position in [(0, -1), (0, 1), (-1, 0), (1, 0), (-1, -1), (-1

            # Get node position
            node_position = (current_node.position[0] + new_position[0], cur

            # Make sure within range
            if node_position[0] > (len(maze) - 1) or node_position[0] < 0 or
                continue

            # Make sure walkable terrain
            if maze[node_position[0]][node_position[1]] != 0:
                continue

            # Create new node
            new_node = Node(current_node, node_position)

            # Append
            children.append(new_node)

        # Loop through children
        for child in children:

            # Child is on the closed list
            if child in closed_list:              # <-- remove inner loop sc
                continue

            # Create the f, g, and h values
            child.g = current_node.g + np.sqrt(np.square(child.position[0] -
            child.h = np.sqrt(np.square(child.position[0] - end_node.positic
            if method=='AStar':
                child.f = child.g + child.h
            elif method=='GBF':
                child.f=child.h
            elif method=='UCS':
```

```
                child.f=child.g


            # Child is already in the open list
            childAlreadyExist=False
            for open_node in open_list:
                if child == open_node and child.g >=open_node.g:
                    childAlreadyExist=True
                    break

            # Add the child to the open list if Chinld not in the open list
            if(not childAlreadyExist):
                open_list.append(child)
def pathLength(path):
    dis=0
    for i in range(len(path)-1):
        x1=path[i][0]
        y1=path[i][1]
        x2=path[i+1][0]
        y2=path[i+1][1]
        dis=dis+np.sqrt(np.square(x1-x2)+np.square(y1-y2))
    return(dis)
# searchMethod: 'AStar' or 'GBF' or 'UCS'
def mazeRunner(start,end,maze,searchMethod='AStar'):
    #force start and end positions to be reachable.
    maze[start[0]][start[1]]=0
    maze[end[0]][end[1]]=0
    expanded_nodes,queue_size,path = InformedSearch(maze, start, end, search
    print("\r\n%s search path length: %f"%(searchMethod,pathLength(path)))
    return(expanded_nodes,queue_size,path)
#########################################################################
print ("#################################################################")
print ("#################################################################")
print ("#################################################################")
# Create random maze environment with controled obstacles
# m x m maze, p
maze = [[0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
        [0, 0, 0, 0, 0, 0, 0, 1, 1, 0],
        [0, 0, 0, 0, 0, 0, 1, 1, 1, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]
start = (0, 0)
end = (9, 9)
# m,p=200,0.02
# maze=createMaze(m,p)
print(maze)
print(mazeRunner(start,end,maze,'AStar'))
print(mazeRunner(start,end,maze,'GBF'))
print(mazeRunner(start,end,maze,'UCS'))
```

```
############################################################
############################################################
############################################################
[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0,
0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0,
0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
[0, 0, 0, 0, 0, 0, 0, 1, 1, 0], [0, 0, 0, 0, 0, 0, 1, 1, 1, 0], [0, 0, 0, 0,
0, 0, 0, 0, 0, 0]]

AStar search path length: 14.485281
(77, 59, [(0, 0), (1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (5, 6), (5, 7),
(5, 8), (6, 9), (7, 9), (8, 9), (9, 9)])

GBF search path length: 15.313708
(13, 29, [(0, 0), (1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (6, 6), (6, 7),
(5, 8), (6, 9), (7, 9), (8, 9), (9, 9)])

UCS search path length: 14.485281
(101, 22, [(0, 0), (0, 1), (0, 2), (0, 3), (1, 4), (2, 5), (3, 6), (4, 7),
(5, 8), (6, 9), (7, 9), (8, 9), (9, 9)])
```

-A* is Optimal because it uses f =g + h, guaranteeing optimal path when heuristic is admissible. -USC is Optimal because it uses f =g, which is the actual cost only, and always explored the lowest cost path first, guaranteeing the shortest path. -GBF is Not Optimal, because it uses f = h, therefore it is heuristic only and ignores the actual cost